

# Dynamic Channel: A Planning Framework for Crowd Navigation

Chao Cao<sup>1</sup>, Peter Trautman<sup>2</sup> and Soshi Iba<sup>2</sup>

**Abstract**—Real-time navigation in dense human environments is a challenging problem in robotics. Most existing path planners fail to account for the dynamics of pedestrians because introducing time as an additional dimension in search space is computationally prohibitive. Alternatively, most local motion planners only address imminent collision avoidance and fail to offer long-term optimality. In this work, we present an approach, called Dynamic Channels, to solve this global to local quandary. Our method combines the high-level topological path planning with low-level motion planning into a complete pipeline. By formulating the path planning problem as graph-searching in the triangulation space, our planner is able to explicitly reason about the obstacle dynamics and capture the environmental change efficiently. We evaluate efficiency and performance of our approach on public pedestrian datasets and compare it to a state-of-the-art planning algorithm for dynamic obstacle avoidance. Completeness proofs are provided in the supplement at <http://caochao.me/files/proof.pdf>. An extended version of the paper is available on arXiv.

## I. INTRODUCTION

Autonomously navigating through crowded human environments is a cornerstone functionality for many mobile robot applications (e.g., service, safety, and logistic robotics). However, crowd navigation requires safe operation of the robot in close proximity to agile pedestrians: a competent crowd planner needs to efficiently generate feasible, optimal, and even socially compliant trajectories. Most existing approaches focus on either global or local optimality. For instance, sampling based motion planners often assume a static environment because adding a time dimension exponentially increases computation. Search-based, optimization-based and geometry-based planning algorithms are subject to the same computational shortcoming. Alternatively, local planners attend to agent dynamics by seeking collision-free paths over short time horizons but neglect global optimality.

We introduce *dynamic channels*, a novel crowd navigation architecture combining global optimality and a heuristic for efficiently managing agent dynamics. Our method abstracts the environment using Delaunay Triangulation and then projects obstacle dynamics onto the triangulation. A modified A\* algorithm then computes the optimal path. Empirically, performance and efficiency are validated with thousands of real world pedestrian datasets. Our approach outperforms state of the art methods by a significant margin for task completion while remaining real-time computable even for large numbers of dynamic obstacles.

<sup>1</sup>Chao Cao is with Robotics Institute, Carnegie Mellon University, Pittsburgh, PA 15213, USA [ccaol@andrew.cmu.edu](mailto:ccaol@andrew.cmu.edu)

<sup>2</sup>Peter Trautman and Soshi Iba with Honda Research Institute, Mountain View, CA 94043, USA [{ptrautman, siba}@honda-ri.com](mailto:{ptrautman, siba}@honda-ri.com)

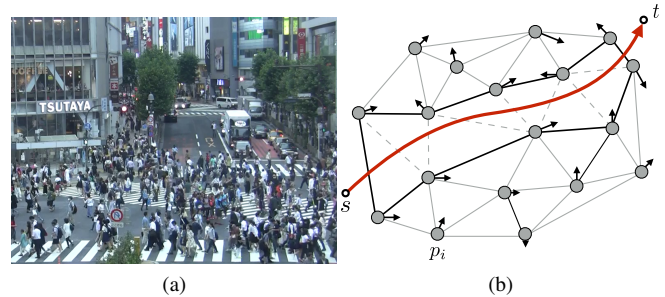


Fig. 1: (a) A crowded crosswalk in Japan, among which the navigation of a robot will be challenging without an efficient planner. (b) The environment abstraction used in our framework. Pedestrians are shown as circles with arrows indicating the moving direction and speed. The "Dynamic Channel" and a safe trajectory inside are shown by bold line segments and the red curve with an arrow.

## A. Local and Global Planning

*Local planning* seeks to optimize near-term objective functions; classic approaches include the Dynamic Window Approach [1], Inevitable Collision States [2], and Velocity Obstacles [3]. Variants of Velocity Obstacles [4], [5], [6], [7] solve the multi-agent system problem for reciprocal collision avoidance by assuming that each agent adopts the same navigation strategy (e.g., each agent passes to the right). However, human behavior is probabilistic, and so assuming fixed actions can lead to unsafe robot behaviors. However, even with guaranteed collision-free local actions, local planning is insufficient, since such trajectories may not guarantee e.g. the shortest traveling distance.

*Global planning* searches for feasible paths connecting initial and goal states of the robot. Most traditional sampling based algorithms including Rapidly-exploring Random Tree (RRT) [8] and Probabilistic roadmap methods (PRM) [9], [10] assume the environment to be static when planning; replanning is employed to account for dynamics. To reduce unnecessary computation [11] proposed a dynamic RRT variant by reusing branches from previous search trees. In [12] a replanning scheme based on RRT\* [13] dealt with unknown dynamic obstacles blocking the robot's path.

Many global planners decouple static and dynamic obstacles. An intuitive strategy relies on a local reactive collision avoidance system to deal with local disturbances. Separating static and dynamic obstacles can also be achieved by hierarchical planning and the combination of potential fields: [14], [15] decouples high and low level planning with a two level-search. Globally, a roadmap is built upon static obstacles while locally a collision free trajectory is obtained. In [16], a dynamic pedestrian potential field selects the traversal and safety optimal trajectory. The same

problem also exists in other planning methods. For Elastic Band [17], Timed Elastic Band [18] and spline-based planners [19], the initial guess determines path partition, which is based on the static assumption. Ultimately, treating static and dynamic obstacles separately has the advantage of being efficient and probabilistically complete. However, sub-optimal paths can result because obstacle dynamics affects global optimality. An important class of planning methods leverages computational geometry to gain efficiency; Voronoi diagrams and Delaunay Triangulation have long been used for environment abstraction. With Voronoi diagrams, the decomposition of free space based on Voronoi cells efficiently generates homotopically distinct paths [20], [21], [22], [23]. Delaunay Triangulation has also been used in graphics to solve the virtual character navigation [24], [25]. These data structures have important complexity properties in 2D planar environments: compared to grid-based maps, computational geometric approaches search a much smaller graph while faithfully capturing topological properties like connectivity. In [26] Triangulation A\* (TA\*) and Triangulation Reduction A\* (TRA\*) is introduced for pathfinding in the triangulation space. Similarly, [27] presents a path planning approach in Constrained Delaunay Triangulation (CDT) to account for static polygon obstacles. Finally [28] uses dynamic Delaunay triangulation (DDT) to solve the problem navigation in the absence of a pre-specified goal position.

However, these algorithms do not reason about obstacle dynamics, and naively adding a time dimension results in exponential computation. Our approach introduces a novel approach to incorporate obstacle dynamics in the global planning step and extends the pathfinding algorithm to efficiently analyze environmental evolution.

### B. Human Aware Navigation

*Human aware navigation* [29] focuses on socially acceptable robot behaviors rather than explicitly solving the collision avoidance problem. In [30] cooperative collision avoidance is leveraged to develop a tractable interacting Gaussian Process (IGP) model. A socially-inspired crowd navigation approach is developed in [31] where the robot follows pedestrians moving towards its goal. However, this method might fail in situations where the major flow of pedestrians is not in the robot's goal direction. Other work leverages learning to model and replicate socially compliant behaviors for crowd navigation [32], [33], [34], [35], [36]

In this work, we do not explicitly address socially acceptable navigation. However, our framework can be extended to incorporate social norms. For example, in the graph construction step (described below), the interaction between people can be encoded in the graph is obtainable (one person taking pictures for the other, a group of friends walking together) so that edges between pedestrians can be denoted as non-crossable for the robot. Moreover, the path generated by our planner naturally captures some social norms. For example, when passing a pedestrian, our planner will favor passing from behind rather than in front of the pedestrian, because the former leads to a shorter traveling distance.

## II. METHODOLOGY

Let the crowd positions at time  $\tau$  be denoted as  $\mathcal{P}_\tau = \{p_i^\tau \mid p_i^\tau \in \mathbb{R}^2, i = 1, \dots, n_\tau\}$ , where  $n_\tau$  is the number of pedestrians. Let  $\mathcal{V}_\tau = \{v_i^\tau \mid v_i^\tau \in \mathbb{R}^2, i = 1, \dots, n_\tau\}$  be pedestrian velocities at  $\tau$ . Let the robot starting point be  $s \in \mathbb{R}^2$  and goal  $t \in \mathbb{R}^2$ . We seek the collision free trajectory that is optimal with respect to travel time and distance.

### A. Delaunay Triangulation

Our first objective is to plan a path through the obstacles  $\mathcal{P}_\tau$  at the topological level. This is achieved by Delaunay triangulation on  $\mathcal{P}_\tau$ , which produces the graph  $\mathcal{T}_\tau$  (light gray graph in Fig. 2c;  $p_i^\tau$  are vertices). The dual graph  $\mathcal{T}_\tau^*$  (black graph in Fig. 2c) has one vertex for each face of  $\mathcal{T}_\tau$  and one edge for face pairs separated by an edge in  $\mathcal{T}_\tau$ . If we define homotopic equivalence as paths that can be continuously deformed into each other without passing through vertices  $p_i^\tau$ , then  $\mathbb{R}^2$ -paths and  $\mathcal{T}_\tau^*$ -graphs are equivalent (Fig 2c, 2d):

*Theorem 2.1:* Any path through  $\mathcal{P}_\tau$  in  $\mathbb{R}^2$  uniquely determines a path in  $\mathcal{T}_\tau^*$ .

*Theorem 2.2:* A path on  $\mathcal{T}_\tau^*$  without repeating vertices corresponds to one homotopy class of paths in  $\mathbb{R}^2$

For brevity, we prove these theorems in a supplement.<sup>1</sup> Thus, instead of optimizing paths in  $\mathbb{R}^2$  we can equivalently search the graph  $\mathcal{T}_\tau^*$ . This graph search is computationally efficient (Section III-B). An important advantage of searching  $\mathcal{T}_\tau^*$  is that we can reason about pedestrian pair interactions, instead of individually checking collisions. Consider a path passing through a group of people as shown in Fig. 2a, and the directional curve showing the robot path. By projecting pedestrian velocities onto triangulation edges, triangulation time evolution is achieved. In this manner, pedestrian dynamics are analyzed as a network.

### B. Dynamic Channel

A path on  $\mathcal{T}_\tau^*$  (Fig. 2c) is equivalent to a triangulated simple polygon called a “channel” [24] (Fig. 2d). We extend this concept to a “dynamic channel.” This dynamic channel is precisely the time evolution of our triangulation.

Pedestrian dynamics cause the dynamic channel to deform constantly. Of most concern is “gate” change (light gray dashed lines in Fig. 2d), the distance between pedestrian  $i$  and  $j$  at time  $\tau$ :

$$D^{ij}(\tau) = \|p_i^\tau - p_j^\tau\| \quad (1)$$

where  $\|\cdot\|$  denotes the Euclidean distance. Let  $D_{thresh} = 2(r_{obs} + S_{safe})$  be a threshold for the width of a gate considered feasible for the robot to pass, where  $r_{obs}$  is the radius of a pedestrian expanded by the radius of the robot, and  $S_{safe}$  is the minimum safe distance between the robot and pedestrians. Solving for

$$D^{ij}(\tau) \geq D_{thresh} \quad (2)$$

<sup>1</sup><http://caochao.me/files/proof.pdf>

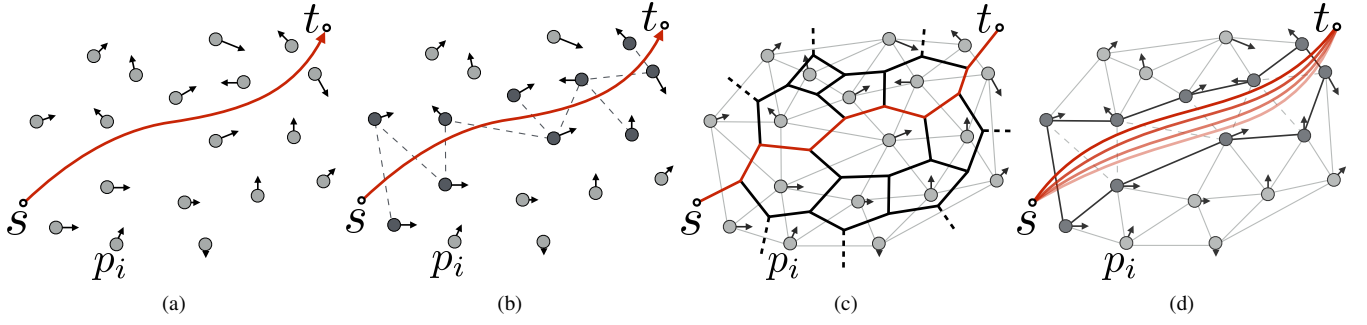


Fig. 2: (a) A path from  $s$  to  $t$  planned through the crowd. Pedestrians are shown as grey circles with arrows denoting velocities. (b) The path sequentially passes through pairs of pedestrians shown as darker circles connected by dashed line segments. (c) The dual graph  $\mathcal{T}^*$  of the triangulation of  $\mathcal{P}$ . A path connecting  $s$  and  $t$  is shown in red. (d) A "channel" (bold line segments) and the corresponding homotopy class of paths (red curves) bounded within.

we obtain the time intervals  $T_{feasible} = [\tau_1, \tau_2] \cup \dots \cup [\tau_{m-1}, \tau_m]$  when the gate is wide enough for the robot to cross. Given the Estimated Time of Arrival (ETA) for the robot to reach the gate  $\tau_{ETA}$  (detailed in the next subsection), safe passage is guaranteed when  $\tau_{ETA} \in T_{feasible}$ . This condition must hold for all gates to guarantee safe passage.

We assume that pedestrian behaviors can be modeled linearly. Thus the trajectory (starting from the current frame  $\tau = 0$ ) of pedestrian  $i$  can be described by:

$$p^i(t) = p_0^i + v^i \tau \quad (3)$$

where  $p_0$  and  $v^i$  are the current position and velocity.  $D^{ij}(\tau)$  is then a quadratic function with respect to  $\tau$  and the graph of  $D^{ij}(\tau)$  is a parabola opening upward as in Fig. 3b. Solving for Equation 2 gives zero, one or two solutions. When there is no solution, the minimum distance  $D_{min}^{ij}(\tau)$  between pedestrians  $i$  and  $j$  will never be too close for the robot to cross. When there is only one solution, a critical moment exists when the robot can be just safe enough to cross. When there are two solutions, denoting  $t_1$  and  $t_2$  with  $t_1 < t_2$ , it is required that  $t_{ETA} \in ([-\infty, t_1] \cup [t_2, +\infty]) \cap [0, +\infty]$  for safe passage as shown in Fig. 3b.

### C. Timed A\* Search

A\* algorithm has been used for searching for shortest paths in the triangulation space [24], [26]. Care needs to be taken for computing the  $g$  and  $h$  values. [27] proposes the "target attractive principle" to determine the placements of nodes when building the dual graph on constrained Delaunay

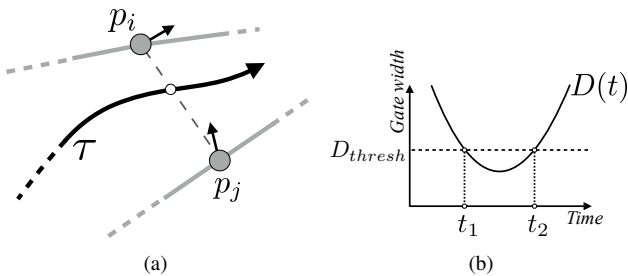


Fig. 3: (a) A gate formed by pedestrians  $p_i$  and  $p_j$ . A path  $\tau$  is planned through the gate. (b) The distance between two pedestrians changes with time as a parabola.  $t_1$  and  $t_2$  show the moments when the gate is wide enough (above the threshold  $D_{thresh}$ ) for safe passage.

Triangulation. Here we adopt the rule-based method as in [27] to determine the graph nodes. Note that in our work, the constraints are introduced by non-crossable edges that will lead to potential collision rather than static obstacles. For compactness, the detailed discussion on node placement is not given here.

Here, we extend the A\* algorithm by incorporating robot and pedestrian dynamics. Within the triangulation framework, we focus on the interaction between connected pedestrian pairs. Thus the dynamics of the robot and pedestrians can be studied by projecting them along the edges in the triangulation. In particular, we compute the  $g$  and  $h$ -value with a lookahead time period. While searching, the velocity of a node representing a triangle can be determined by interpolating the velocities of three endpoints, which gives  $g(\tau), h(\tau)$ . We then compute  $\tau_{ETA}$  to determine  $g(\tau_{ETA})$  and  $h(\tau_{ETA})$ . Similar to the estimate of  $g$  value in a regular A\* algorithm, we track path history while searching. For each node, the current path to the node is time parameterized by taking into account the kinematics and dynamics of the robot and curvature of the path. Note that due to the approximated node placements at searching time as in [27] and the linear model assumption,  $\tau_{ETA}$  can only be approximated accurately in a finite time horizon. Due to the time-variant  $g(\tau)$  and  $h(\tau)$  value, our planner will seek a candidate shortest path. For instance, when passing a walking pedestrian, our planner will passing behind rather than in front of, even though in front of is currently a shorter path.

### D. Path Optimization

The output from our Timed A\* algorithm is a path on  $\mathcal{T}^*$  consisting of a series of triangles that form a channel. We find the shortest path in the channel that ensures sufficient clearance. *Funnel algorithm* [37], [38], [39] can be used for computing a shortest path within a simple triangulated polygon connecting the start and goal points. To account for non-zero radius obstacles, [26] proposed a modified *funnel algorithm* by inserting conceptual circles at each vertex of the channel. The resulting path is composed of alternating straight-line segments and arcs. In our work, we further extend this method to account for the pedestrian dynamics. Given a pedestrian  $p_i^T$ , we project the velocity  $v_i^T$  onto the vector  $e_{ji} = p_j - p_i$ , where  $p_j$  is the other end of the edge

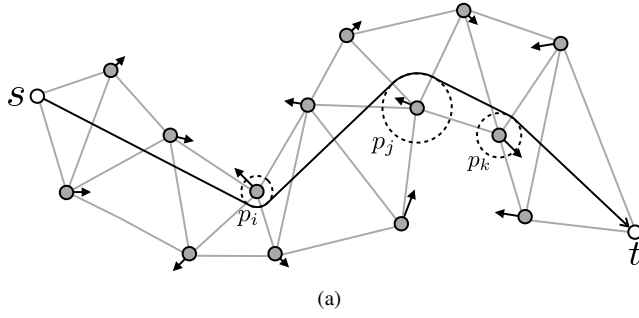


Fig. 4: Within a channel the shortest path is determined by our *funnel algorithm*. Clearance radius is determined by whether the pedestrian is “threatening” or “enlarging” the channel. Larger radius is assigned for the former ( $p_j$ ) while smaller radius is assigned to the latter ( $p_i, p_k$ ).

in the channel. If the projection  $v'_i > 0$ , the path will pass by  $p_i^T$  in the front, potentially blocking  $p_i^T$ 's moving direction. In this case, we assign a circle with the radius  $r = k||v'_i||$  around  $p_i^T$  to create clearance, where  $k$  is a tunable parameter. Similarly, if  $v'_i < 0$ , the path will pass from behind. Figure 4a provides an illustration, where pedestrians  $p_i$ ,  $p_j$  and  $p_k$  are assigned different sized circles based on their velocities.

Most of the collision avoidance effort described above is implicitly done by enforcing timely traversal. However, when obstacles move faster than the robot simply following the path cannot guarantee collision-free navigation. In such cases, a low-level robot-specific controller is needed to perform maneuvers for the robot to avoid collision. In a Delaunay Triangulation, the circumcircle of any triangle will not include other points inside. Thanks to this property, a collision-free circular area can be identified in the robot's vicinity, which can be used for local maneuvers when needed. Furthermore, if no paths are found by the Timed A\* algorithm, other behavioral strategies can be employed to utilize the local collision-free area. For example, the robot can follow the people walking in the goal direction while maintaining a proper clearance to others as in [31].

### III. EXPERIMENTAL EVALUATION

We demonstrate the advantage of our planner by testing on a large number of navigation tasks in simulated environments based on public human-trajectory datasets. We compare success rate, efficiency of traversal, and time complexity of our algorithm against two other planning methods. Finally, we empirically study the effect of observability on our algorithm.

#### A. Experiment Settings

1) *Datasets*: For experiments, we implemented our planning framework to control a car-like robot navigating in simulated environments. In particular, we use the pure-pursuit algorithm [40] to track a path output from our planner. For benchmarking, in our simulation we replay the human-trajectories from two public datasets: ETH [41] and UCY [42]. There are five subsets in total recorded in different scenarios, denoted as *eth\_hotel*, *eth\_univ*, *ucy\_univ*, *ucy\_zara01* and *ucy\_zara02*.

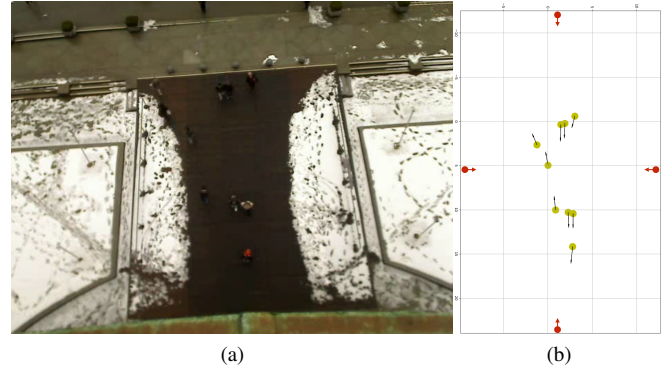


Fig. 5: Experiment Settings. (a) shows a frame from the ETH dataset. (b) shows the pedestrian positions extracted from the frame. Red circles with arrows show the starting positions of the robot in the experiments.

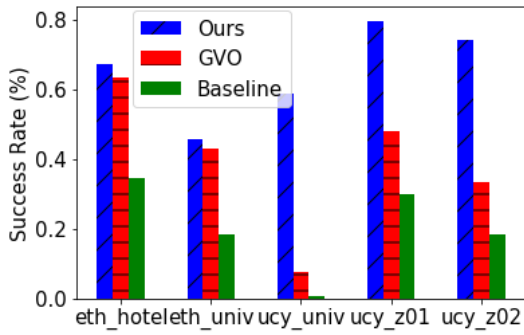
For each dataset, the trajectory data are first interpolated and then re-projected back to the world coordinate using the homography matrices provided. We define a rectangular region as the workspace which is bounded by extreme values of the pedestrian positions of each dataset. For the navigation task, the robot starts at four mid-points of the workspace boundaries and moves to the antipodal goal position as shown in fig 5. In the experiments, only pedestrians are considered to be obstacles and all free space is assumed traversable for the robot. Fig. 5b shows an example testing configuration, where pedestrians are shown as yellow circles with black arrows indicating velocity. The starting positions are shown as red circles with arrows. The velocities of pedestrians are approximated by the position differences between two consecutive frames divided by the sampling period.

Trials begin at a sequence of starting times at intervals of 3s, which results in 1520, 1148, 184, 148, and 120 trials for each dataset respectively. All experiments were run on a 2.6 GHz CPU/15.5 GB memory computer.

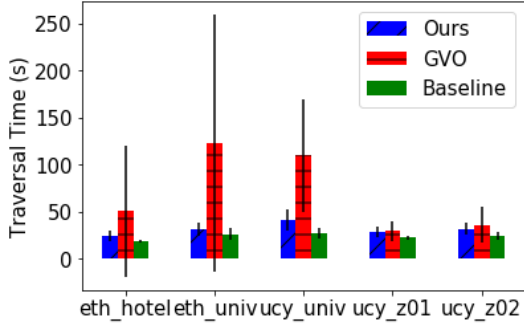
2) *Generalized Velocity Obstacle Planner*: For comparison, we implemented a planning algorithm based on the Generalized Velocity Obstacle (GVO, [43]) to control the same car-like robot. The algorithm first randomly samples a control input  $u$  from a feasible set. Then a trajectory is simulated within a time horizon for the input, which takes into account the robot dynamics. The trajectory is then checked for collision with GVO prediction on each moving agent. Finally, among all collision-free control inputs, the one closest to the *preferred* control  $u^*$  is chosen. As suggested in the paper, we choose the *preferred* control to be the one driving the robot directly to the goal as if there is no obstacles. However, we do not explicitly solve the optimization problem for  $t^*$  that results in minimum distance between the robot trajectory and obstacles. Instead, we discretize the trajectory and check collision at 10Hz. Only in this manner can we run the algorithm efficiently enough for the benchmarking. The other parameters used were *time\_horizon* = 3.5s, *wheelbase* = 1m and *sampling\_per\_timestep* = 40.

3) *“Wait-and-go” Planner*: We designed the simplest possible crowd navigation strategy, “wait-and-go”, as a base-





(a) Success Rate



(b) Efficiency

Fig. 6: Experiment Results. Comparison of: (a) the success rate of navigation tasks in each dataset. (b) The average traveling time of the robot in each dataset. Vertical line segments show the variance by a standard deviation.

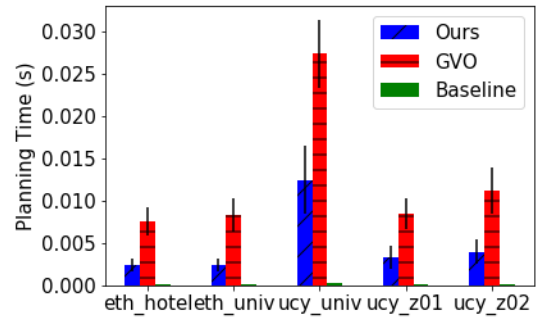
line: the robot drives in a straight-line towards the goal. When the robot comes too close to pedestrians or there is a potential collision determined by Velocity Obstacle, the robot stops (wait) and resumes moving (go) when possible.

### B. Comparing success rate, efficiency and time complexity

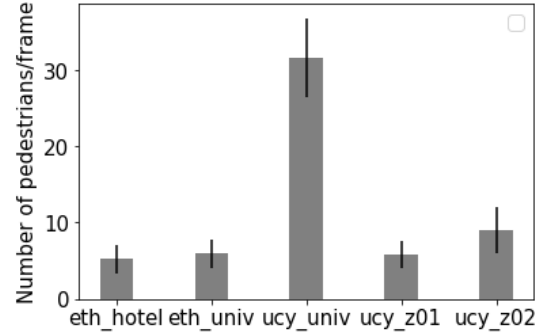
For each trial, the first frame is concatenated to the last one to form a cycle. We run each trial until the robot reaches the goal position or all the frames have run out. If the distance between the robot and any pedestrian is less than  $1m$ , a collision is reported and the trial is failed. We also calculated running time to compare efficiency of the three approaches. Note that when a trial fails, the timer will not stop until the trial is finished. The robot has a speed limit of  $1.2m/s$  which is slightly slower than the average human walking speed.

1) *Success Rate*: From the result shown in Fig. 6a, our planner has the highest success rates in all five datasets, while GVO ranks second and the baseline planner performs last. For the challenging dataset *ucy\_univ*, where the crowd density reaches 30 pedestrians per frame, our planner significantly outperforms the baselines. Anecdotaly, most of the failure cases are due to sudden pedestrian appearance and nonlinear pedestrian movement.

2) *Traveling Efficiency*: For traversal efficiency, as shown in Fig. 6b, our planner is more efficient than "wait-and-go," while GVO exhibits much longer times with much larger variance. This is because GVO tends to passively dodge approaching pedestrians (resulting more evasive strategies),



(a) Time Complexity



(b) Number of Pedestrians

Fig. 7: Experiment Results. Comparison of: (a) the average planning time of a frame for each dataset. (b) The average number of pedestrians in each dataset. Vertical line segments show the variance by a standard deviation.

while our planner actively looks for open spaces and a long-term shortest route.

3) *Computational Complexity*: For computational complexity (7a), both our approach and GVO sees an increase in computation as more pedestrians enter the scene. However, our method is significantly more time efficient than GVO.

Qualitatively, Fig. 8 shows a sequence of the robot navigating in the *eth\_hotel* dataset. At the beginning, the robot plans to pass by the pedestrians from the right. Between  $t = 7.25$  and  $t = 11.25$ , there suddenly appears a group of people on the right side. Our planner is able to adapt to this and steer the robot to the left for a safe-pass.

### C. Observability Effects

We also study observability effects by testing our planner on a larger simulated scene with limited sensor ranges. The simulated pedestrians are generated as follows. For every  $5s$ , 2 pedestrians are spawned from the two sides of the workspace. Each pedestrian is assigned with a random goal position at the other side and a random linear velocity towards the goal. While traveling to the goal the pedestrians will randomly change their goal and speeds every  $3s$ . The observation range of the robot is tested from  $1m$  to  $30m$ , with the interval being  $1m$  when under  $10m$  and  $5m$  when above  $10m$ . We also run trials without sensor range limit for comparison. Beyond the sensing range, only the goal position is known, so the robot has no information about pedestrians outside the field of view. In each trial, the robot navigates from the lower right corner to the upper left corner

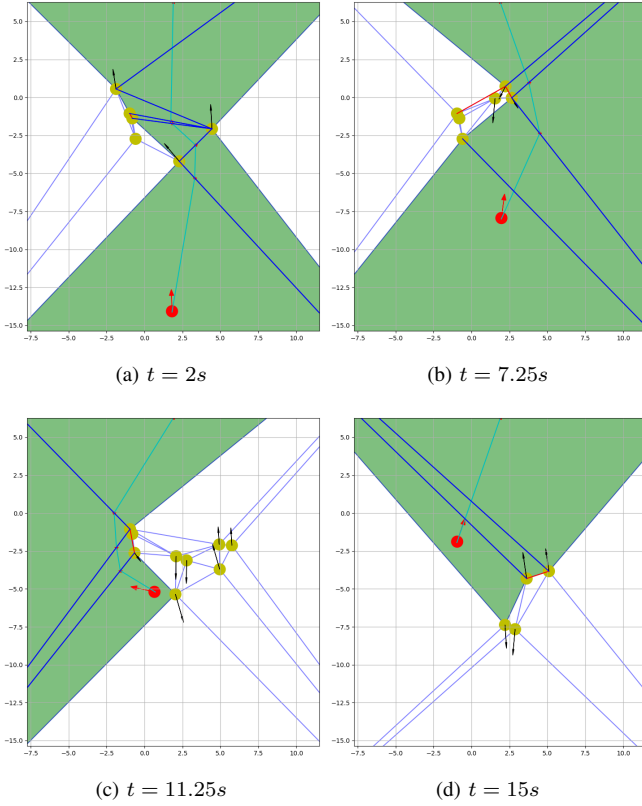


Fig. 8: The path found by our planner in the *eth\_univ* dataset. Green areas show dynamic channel. Cyan line shows trajectory. Blue line segments show valid gates to cross; red ones show unsafe gates. The robot (red filled circle) attempts to navigate from bottom to top. Four static pedestrians inhabit the four corners of the workspace, allowing robot to travel to the crowd's side.

or reversely. For each sensing range, 100 trials are performed. Fig. 9 shows an example scene when the robot navigates in the synthetic crowd with the sensing range limited to 30m.

Results are presented in Fig. 10. In Fig. 10a and Fig. 10b, we observe that the sensor range does not make a big difference for the success rate and traveling time beyond 3m. Medium ranges (3 to 9m) are only slightly better than long ranges (10 to without limit). This is because the longer the channel, the less accurate the  $\tau_{ETA}$  will be, and thus the guarantee for safe-passage for later portions of the channel are less likely to be valid. A practical solution to this problem is to set a time horizon for the planner. Whenever the estimated  $\tau_{ETA}$  is longer than the horizon, we release the safety check. For the time complexity in Fig. 10c, we see that the time needed for a planning cycle increases rapidly with the increasing number of pedestrians per frame as in Fig. 10d. However, our algorithm is still efficient in such crowded environments. It can be seen from the figure that for pedestrian number over 50, our method re-plans at 50Hz and with less than 30 pedestrians our method re-plans at 100Hz.

#### IV. CONCLUSIONS AND FUTURE WORK

We introduced a geometry-based planning framework for crowd navigation. Our primary contribution is efficient global

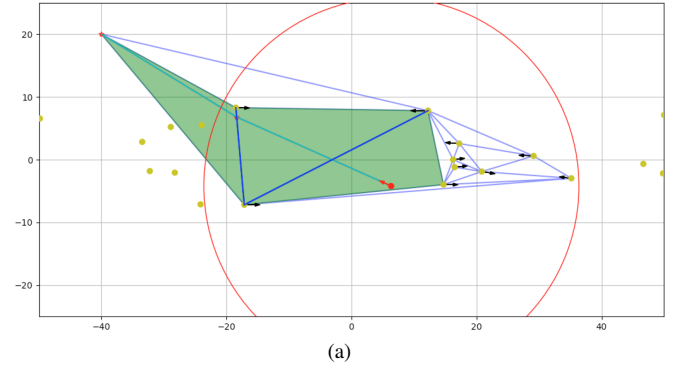


Fig. 9: Path found by our planner in a synthetic dataset with a limited sensor range, denoted by the red circle. Pedestrians outside the sensor range are not included in the triangulation thus not observed by the planner.

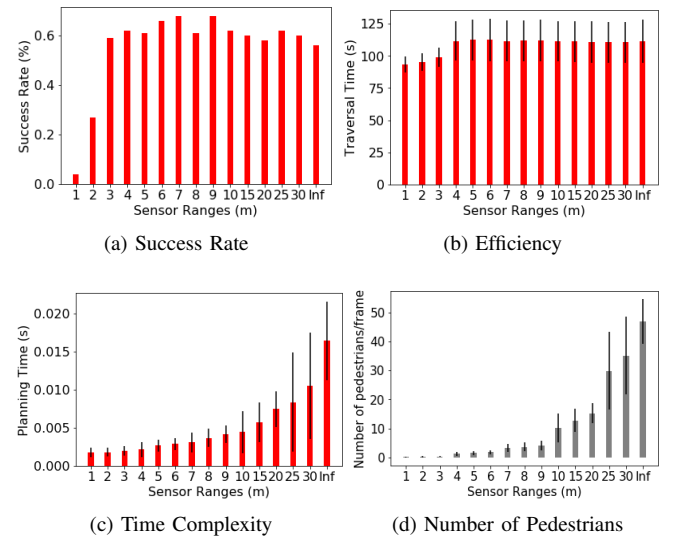


Fig. 10: Experiment Results on the effect of observability. Comparison of: (a) average navigation success rates, (b) average traveling time, (c) average planning time and (d) average number of pedestrians seen by the planner with different sensing ranges. Vertical line segments show standard deviation.

planning while accounting for obstacle dynamics; a proof of completeness and optimality is provided. Empirically, dynamic channels improved success rate by up to a factor of six in the most crowded environments, task completion by up to a factor of three, and decreased computational burden by a factor of two. However, our framework has limitations. First, the method assumes perfect measurements. Second, we assume linear pedestrian motion models; even though our method was evaluated on real world pedestrian datasets with extended periods of nonlinear motion, further experiments are needed to validate robustness in interactive crowds. Future work will integrate dynamic channels with low-level planners to increase this robustness. Additionally, we will extend our architecture for multi-robot navigation, crowd simulation, and the handling of both dynamic and static obstacles with arbitrary shapes and movement uncertainty.

## REFERENCES

- [1] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics and Automation Magazine*, 1997.
- [2] T. Fraichard and H. Asama, "Inevitable Collision States: A Step Towards Safer Robots?" in *ICIRS*, 2003.
- [3] P. Fiorini and Z. Shiller, "Motion Planning in Dynamic Environments using Velocity Obstacles," *International Journal of Robotics Research*, 1998.
- [4] J. van den Berg, S. Guy, M. Lin, and D. Manocha, "Reciprocal n-body Collision Avoidance," in *ISRR*, 2009.
- [5] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Optimal Reciprocal Collision Avoidance for Multi-Agent Navigation," p. 8, 2010.
- [6] J. Snape, J. van den Berg, S. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Transactions on Robotics*, 2011.
- [7] J. Van Den Berg, J. Snape, S. J. Guy, and D. Manocha, "Reciprocal collision avoidance with acceleration-velocity obstacles," *IEEE International Conference on Robotics and Automation*, 2011.
- [8] J. Kuffner and S. LaValle, "RRT-connect: An efficient approach to single-query path planning," *IEEE International Conference on Robotics and Automation*, 2000.
- [9] L. Kavraki, L. Kavraki, P. Svestka, P. Svestka, J.-C. Latombe, J.-C. Latombe, and M. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *Robotics and Automation*, 1996.
- [10] L. E. Kavraki, M. N. Kolountzakis, and J.-c. Latombe, "Analysis Of Probabilistic Roadmaps For Path Planning," *IEEE Transactions on Robotics and automation*.
- [11] M. Zucker, J. Kuffner, and M. Branicky, "Multipartite RRTs for rapid replanning in dynamic environments," *IEEE International Conference on Robotics and Automation*, 2007.
- [12] D. Connell and H. M. La, "Dynamic Path Planning and Replanning for Mobile Robots using RRT\*," 2017. [Online]. Available: <http://arxiv.org/abs/1704.04585>
- [13] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," in *IJRR*, 2011.
- [14] Jur P. van den Berg and Mark H. Overmars, "Roadmap-Based Motion Planning in Dynamic Environments," *Transactions on Robotics*, 2005.
- [15] J. Van Den Berg and M. Overmars, "Kinodynamic motion planning on roadmaps in dynamic environments," *IEEE International Conference on Intelligent Robots and Systems*, 2007.
- [16] M. Svenstrup, T. Bak, and J. Andersen, "Trajectory Planning for Robots in Dynamic Human Environments," in *IROS*, 2010.
- [17] S. Quinlan and O. Khatib, "Elastic bands: connecting path planning and control," [1993] *Proceedings IEEE International Conference on Robotics and Automation*, pp. 802–807, 1993.
- [18] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Efficient trajectory optimization using a sparse model," *2013 European Conference on Mobile Robots*.
- [19] B. Lau, C. Sprunk, and W. Burgard, "Kinodynamic motion planning for mobile robots using splines," *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2427–2433, 2009.
- [20] H. Choset and J. Burdick, "Sensor-based exploration: The hierarchical generalized Voronoi graph," *International Journal of Robotics Research*, vol. 19, no. 2, pp. 96–125, 2000.
- [21] M. Garber and M. Lin, "Constraint-based motion planning using voronoi diagrams," *Algorithmic Foundations of Robotics V*, 2004.
- [22] B. Banerjee and B. Chandrasekaran, "A framework of Voronoi diagram for planning multiple paths in free space," *Journal of Experimental & Theoretical Artificial Intelligence*, 2013.
- [23] M. Kuderer, C. Sprunk, H. Kretzschmar, and W. Burgard, "Online generation of homotopically distinct navigation paths," *IEEE International Conference on Robotics and Automation*, 2014.
- [24] M. Kallmann, "Path planning in triangulations," *International Joint Conference on Artificial Intelligence*, pp. 49–54, 2005.
- [25] P. Brož, M. Zemek, I. Kolingerová, and J. Szkandera, "Dynamic Path Planning With Regular Triangulations," vol. 24, no. 3, pp. 119–142, 2014.
- [26] D. Demyen and M. Buro, "Efficient triangulation-based pathfinding," *Proceedings of the 21st National Conference on Artificial Intelligence*, vol. 6, pp. 942–947, 2007.
- [27] H. Yan, H. Wang, Y. Chen, and G. Dai, "Path planning based on Constrained Delaunay Triangulation," *Intelligent Control and Automation, 2008. WCICA 2008. 7th World Congress on*, pp. 5168–5173, 2008.
- [28] J. Chen, C. Luo, M. Krishnan, M. Paulik, and Y. Tang, "An enhanced dynamic Delaunay triangulation-based path planning algorithm for autonomous mobile robot navigation," 2010.
- [29] T. Kruse, R. Alami, A. K. Pandey, and A. Kirsch, "Human-aware robot navigation: a Survey," in *RAS*, 2013.
- [30] P. Trautman and A. Krause, "Unfreezing the robot: Navigation in dense, interacting crowds," *IEEE International Conference on Intelligent Robots and Systems*, 2010.
- [31] J. Müller, C. Stachniss, K. O. Arras, and W. Burgard, "Socially Inspired Motion Planning for Mobile Robots in Populated Environments," *Computer*, 2008.
- [32] M. Kuderer, H. Kretzschmar, and W. Burgard, "Teaching Mobile Robots to Cooperatively Navigate in Populated Environments," in *IROS*, 2013.
- [33] C. I. Mavrogiannis, V. Blukis, and R. A. Knepper, "Socially Competent Navigation Planning by Deep Learning of Multi-Agent Path Topologies," in *IROS*, 2017.
- [34] M. Luber, L. Spinello, J. Silva, and K. O. Arras, "Socially-Aware Robot Navigation: A Learning Approach," in *IROS*, 2012.
- [35] P. Henry, C. Vollmer, B. Ferris, and D. Fox, "Learning to Navigate Through Crowded Environments," in *ICRA*, 2010.
- [36] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially Aware Motion Planning with Deep Reinforcement Learning," in *IROS*, 2017.
- [37] B. Chazelle, "A theorem on polygon cutting with applications," in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982.
- [38] D. T. Lee and F. P. Preparata, "Euclidean shortest paths in the presence of rectilinear barriers," *Networks*, vol. 14, no. 3, pp. 393–410, 1984.
- [39] J. Hershberger and J. Snoeyink, "Computing minimum length paths of a given homotopy class," *Lecture Notes in Computer Science*, 1991.
- [40] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie-Mellon UNIV Pittsburgh PA Robotics INST, Tech. Rep., 1992.
- [41] S. Pellegrini, A. Ess, K. Schindler, and L. Van Gool, "You'll never walk alone: modeling social behavior for multi-target tracking," in *International Conference on Computer Vision*, 2009.
- [42] A. Lerner, Y. Chrysanthou, and D. Lischinski, "Crowds by example," *Computer Graphics Forum*, vol. 26, no. 3, pp. 655–664, 2007.
- [43] D. Wilkie, J. Van Den Berg, and D. Manocha, "Generalized velocity obstacles," *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2009*, no. June 2014, pp. 5573–5578, 2009.